

Course 3: Unsupervised Learning



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

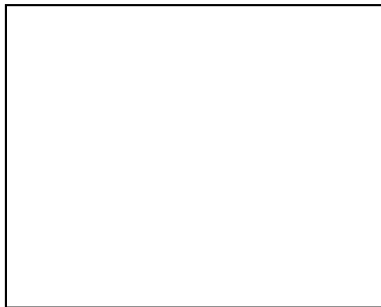
Last session

- 1 Supervised learning - learning from labeled examples
- 2 Challenges of supervised learning
- 3 Evaluation: ground truth and metrics

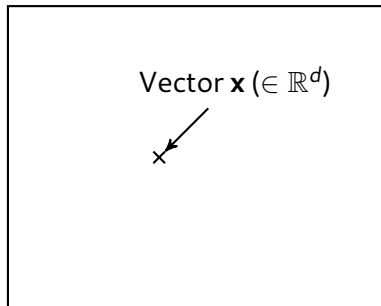
Today's session

- 1 Learning from unlabeled examples
 - 1 Clustering
 - 2 Decomposition
 - 3 Manifold learning
- 2 Evaluating without labels
- 3 A note on data cleaning

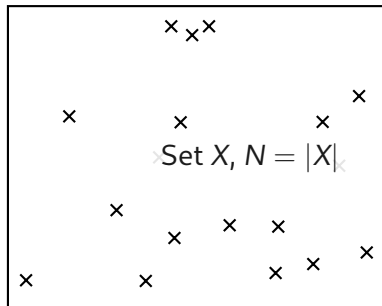
Vector space ($\mathbb{R}^d$)



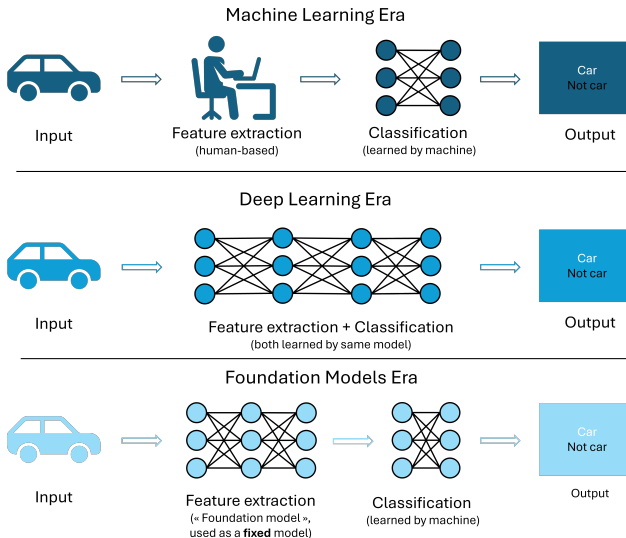
Vector space ($\mathbb{R}^d$)



Vector space ($\mathbb{R}^d$)



What is the vector x ?



(Similar to last session)

Unsupervised learning

Goal

Discover patterns/structure in X ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction,
 - Quantization
 - Visualization...



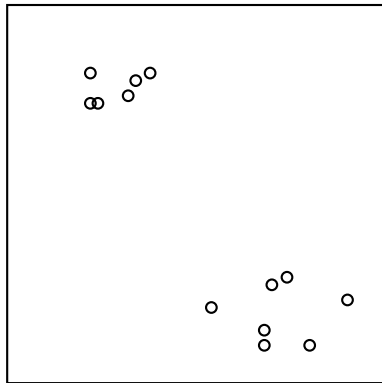
Unsupervised learning

Goal

Discover patterns/structure in X ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction, Quantization
 - Visualization...



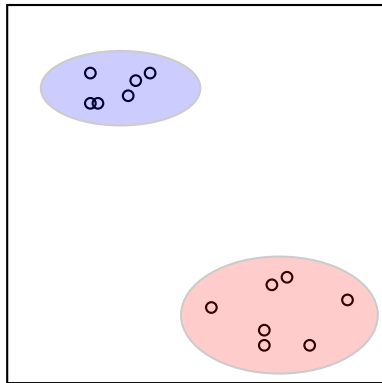
Unsupervised learning

Goal

Discover patterns/structure in X ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction, Quantization
 - Visualization...



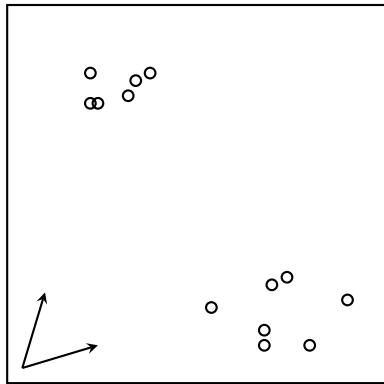
Unsupervised learning

Goal

Discover patterns/structure in X ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction, Quantization
 - Visualization...



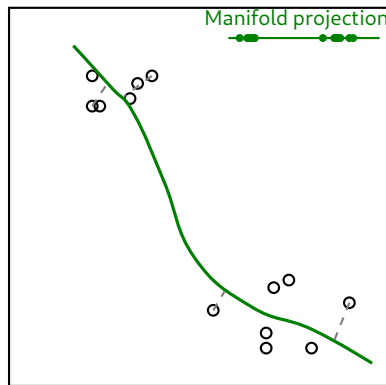
Unsupervised learning

Goal

Discover patterns/structure in X ,

Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction,
 - Quantization
 - Visualization...



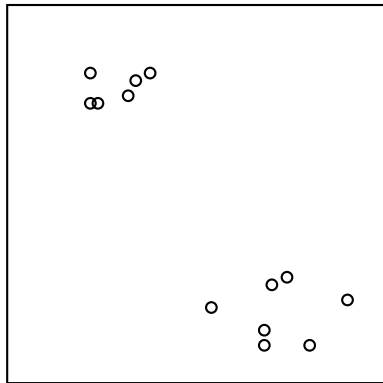
Unsupervised learning

Goal

Discover patterns/structure in X ,

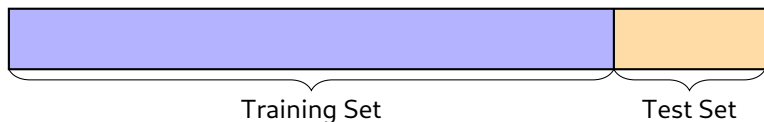
Unsupervised learning

- Unsupervised = no expert, no labels
- Main approaches:
 - Clustering = find a partition of X in K subsets,
 - Decomposition using K vectors.
 - Manifold Learning.
- Applications :
 - Dimensionality reduction, Quantization
 - Visualization...

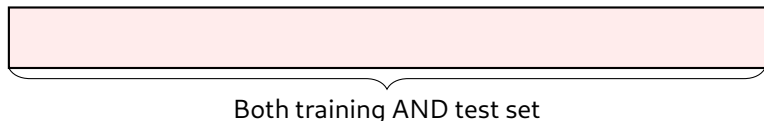


Training and Test sets?

Full Dataset in train/test sets (as for supervised):



...Or actually:

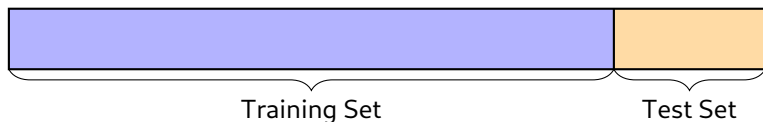


The same training and test sets??

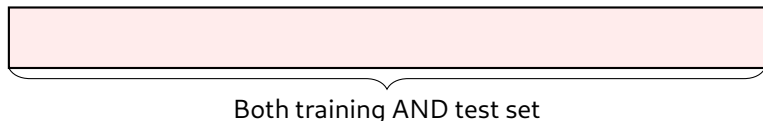
Because unsupervised learning does not rely on external annotations, the "training" and "test" settings are not relevant (**there is no "correct answer" to learn and generalize!**).

Training and Test sets?

Full Dataset in train/test sets (as for supervised):



...Or actually:



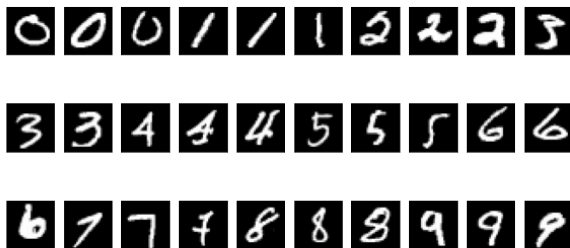
The same training and test sets??

Because unsupervised learning does not rely on external annotations, the “training” and “test” settings are not relevant (**there is no “correct answer” to learn and generalize!**).

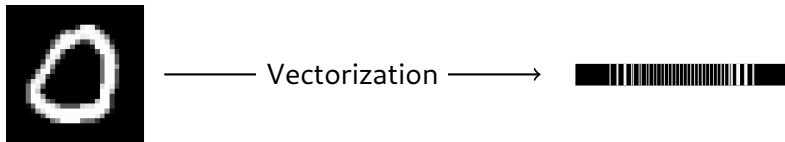
A classical dataset: MNIST dataset (1/2)

MNIST Dataset

- "Toy" dataset (=small and easy)
- 60000 + 10000 handwritten digits



A classical dataset: MNIST dataset (2/2)

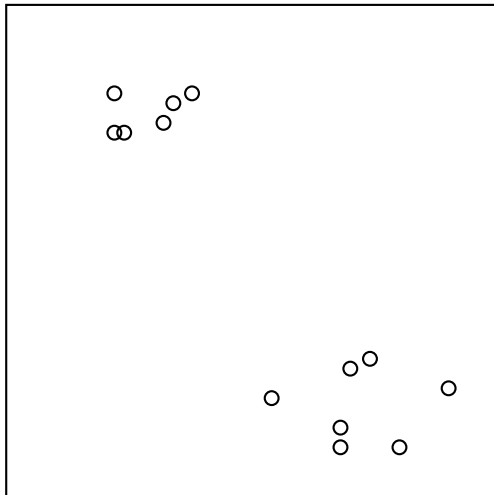


Hence, all images are interpreted as 1D vectors!

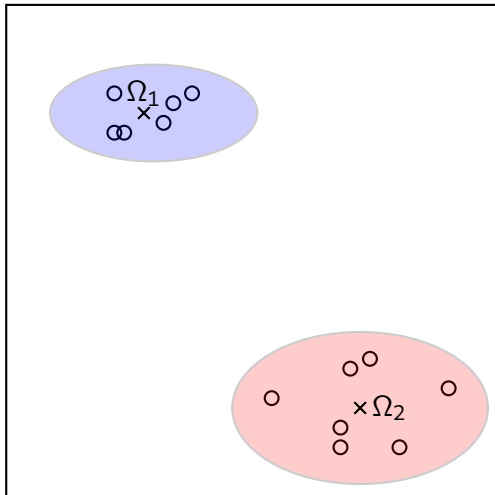
Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

Clustering: the intuition



Clustering: the intuition



A formal definition of clustering

An example to perform clustering is to rely on distances to centroids. We define K cluster centroids $\Omega_k, \forall k \in [1..K]$. Here, each vector is associated with the cluster whose centroid is of minimal distance.

Definitions

We denote $q : \mathbb{R}^d \rightarrow [1..K]$ a function that associates a vector $\mathbf{x}$ with the index of (one of) its closest centroid $\Omega_{q(\mathbf{x})}$. Formally:

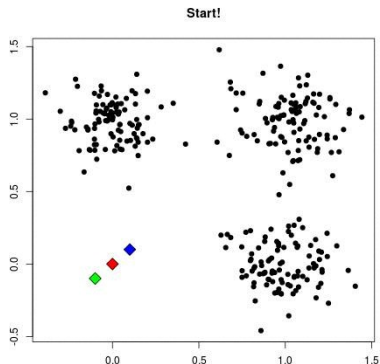
- $\forall k \in [1..K], \Omega_k \in \mathbb{R}^d$
- $\forall \mathbf{x} \in X, \forall j \in [1..K], \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2 \leq \|\mathbf{x} - \Omega_j\|_2$
- Error (inertia) $E(q) \triangleq \sum_{\mathbf{x} \in X} \|\mathbf{x} - \Omega_{q(\mathbf{x})}\|_2^2$
- $X = \bigcup_k \underbrace{\{\mathbf{x} \in X, q(\mathbf{x}) = k\}}_{\text{cluster } k}$

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



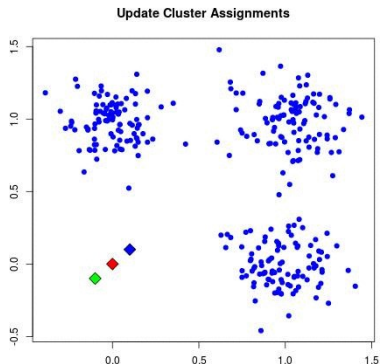
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



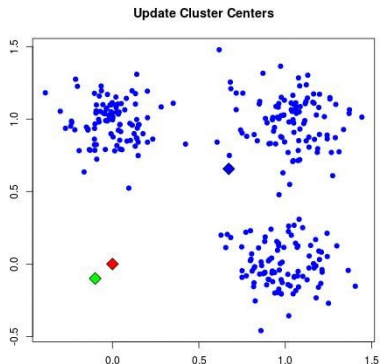
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



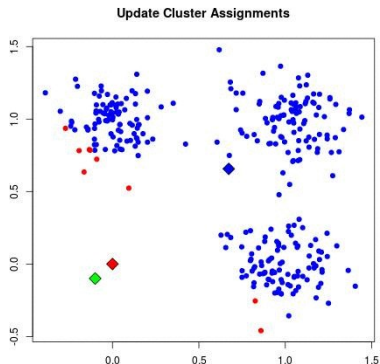
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



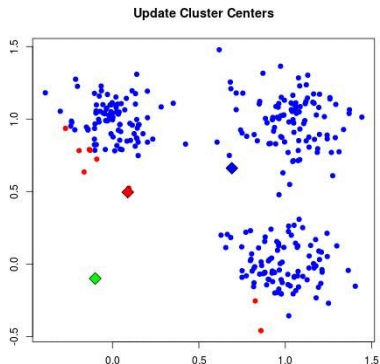
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



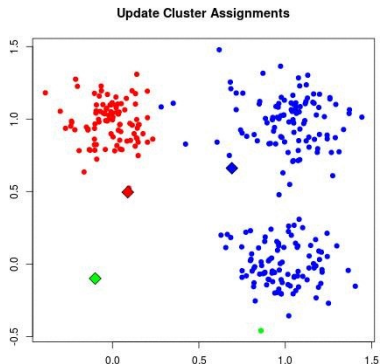
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



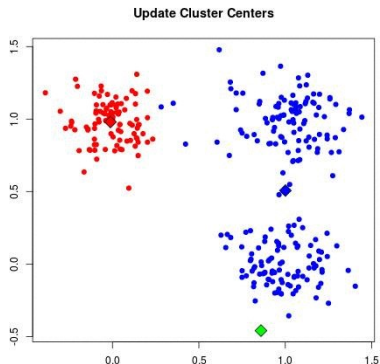
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



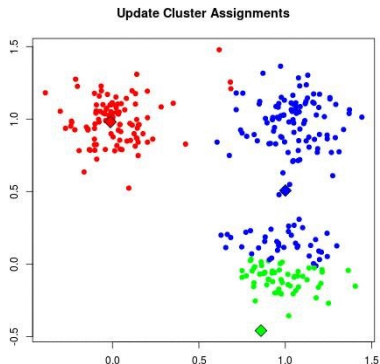
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



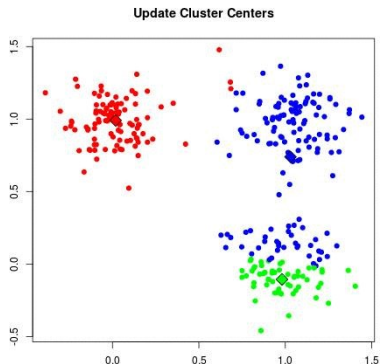
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



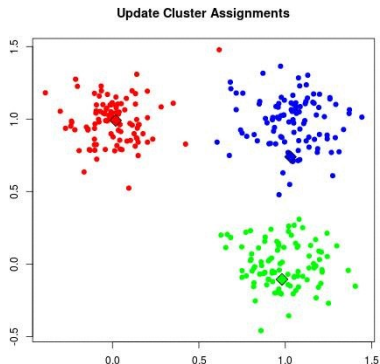
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



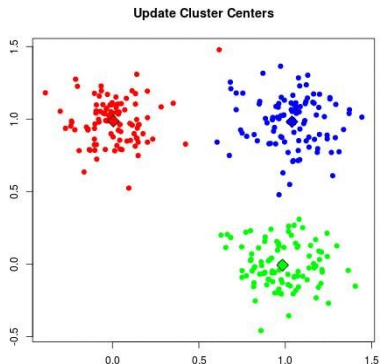
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat .



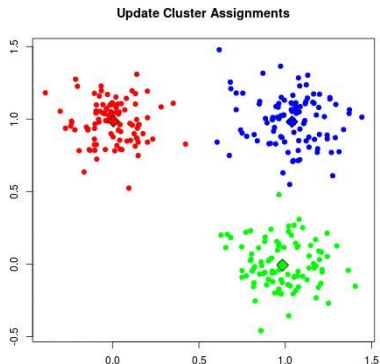
Reference: <https://mubaris.com/posts/kmeans-clustering/>

A standard algorithm: K-Means

K-means algorithm

First: initialize K cluster centroids.

- 1 Assign each data point to the cluster of closest centroid.
- 2 Compute the new centroids as the average of the data points in each cluster.
- 3 Repeat until convergence.



Reference: <https://mubaris.com/posts/kmeans-clustering/>

In practice: K-Means on MNIST

Clustering MNIST

Using K -means algorithm with $K = 10$.

→ Clusters capture, to some extent, the different digits.

Original MNIST examples

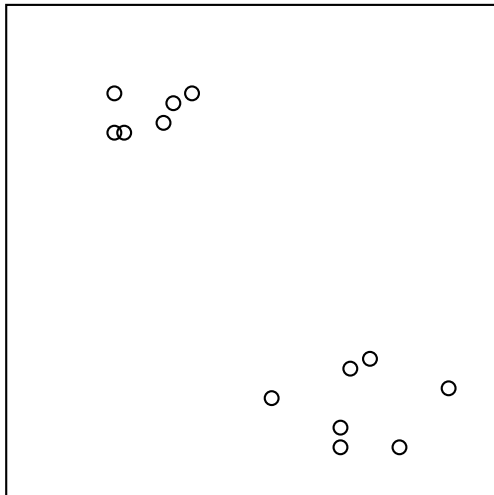


Cluster centroids

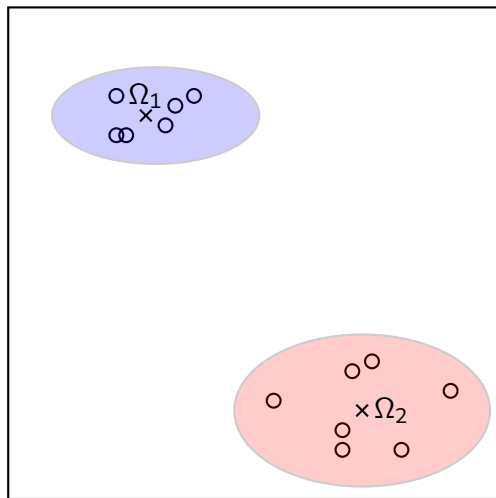


Note: we recall that images are vectorized for the clustering to make sense!
They are only displayed in 2D to be interpretable.

Impact of the choice of K

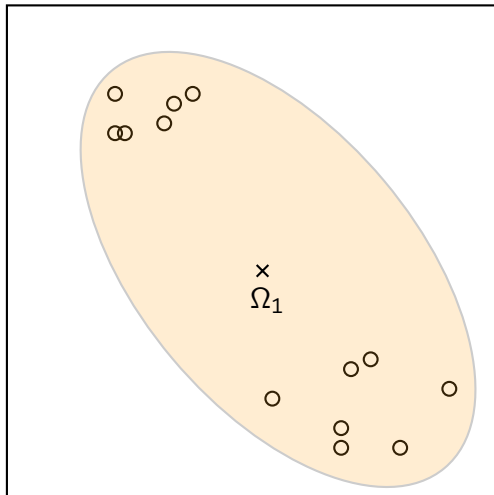


Impact of the choice of K



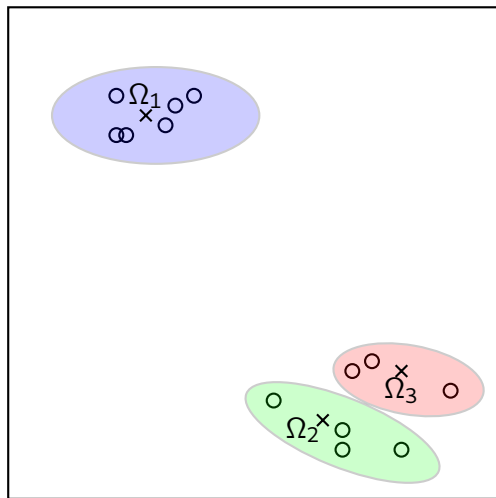
$K = 2$

Impact of the choice of K



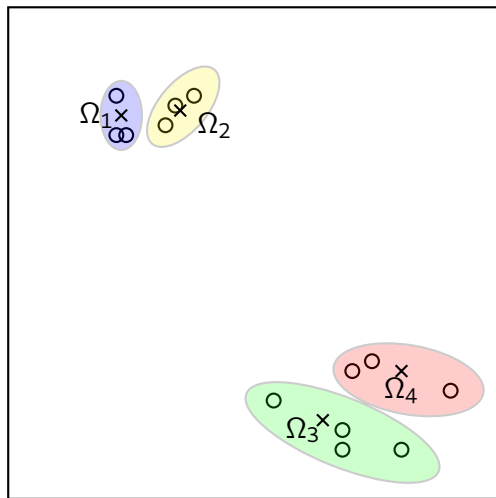
$K = 1$

Impact of the choice of K



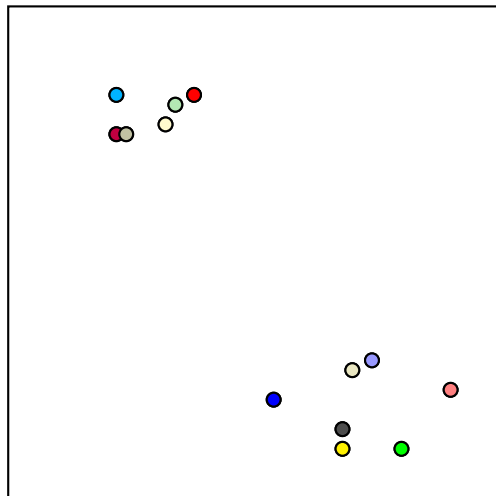
$K = 3$

Impact of the choice of K



$K = 4$

Impact of the choice of K



$$K = N$$

(each data point is its own centroid)

Choosing K

Impact of K

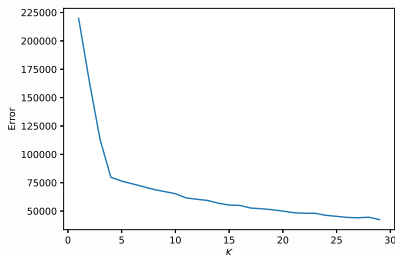
Changing the number of centroids changes the clustering...

→ **And the signification of clusters!**

Choosing K : compromise between **error** and **compression** ($\kappa = 1 - \frac{K}{N}$: the fewer clusters, the more we compress).

"Elbow" heuristic

Not always clear... May be a good starting point!



Choosing K

Impact of K

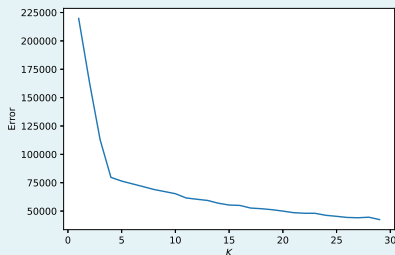
Changing the number of centroids changes the clustering...

→ **And the signification of clusters!**

Choosing K : compromise between **error** and **compression** ($\kappa = 1 - \frac{K}{N}$: the fewer clusters, the more we compress).

"Elbow" heuristic

Not always clear... May be a good starting point!



Optimal clustering is not tractable!

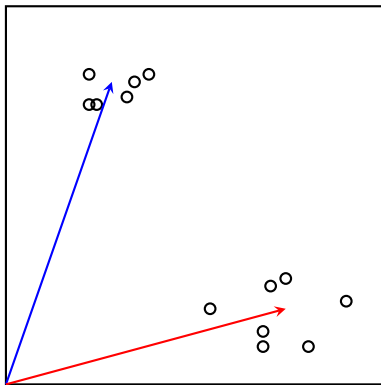
Optimal clustering

- Optimal clustering: $q^* \triangleq \arg \min_{q: \mathbb{R}^d \rightarrow [1..K]} E(q)$, and $E_{opt_K} \triangleq E(q^*)$,
- By construction, increasing K can only decrease the optimal error:
 $0 = E_{opt_N} \leq E_{opt_{N-1}} \leq \dots \leq E_{opt_1} = N \cdot \text{var}(X)$
- Finding an optimal clustering is an NP-hard problem!

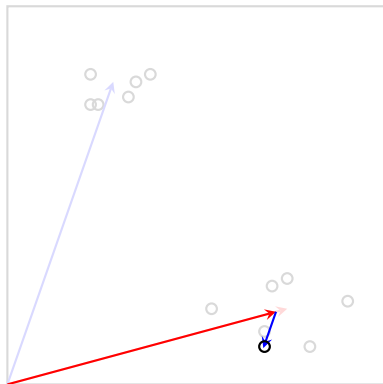
Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

Decomposition

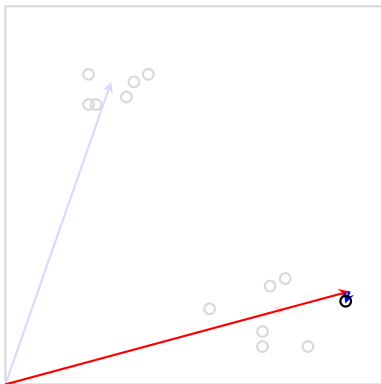


Decomposition



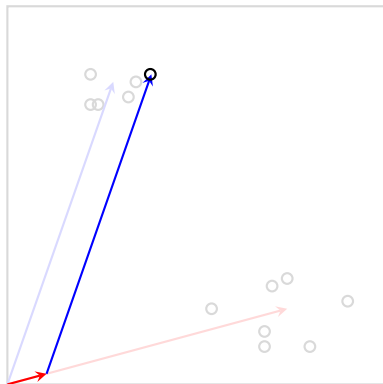
$$\begin{aligned} \mathbf{x} &= 0.96 \times \text{red arrow} \\ &+ -0.12 \times \text{blue arrow} \end{aligned}$$

Decomposition



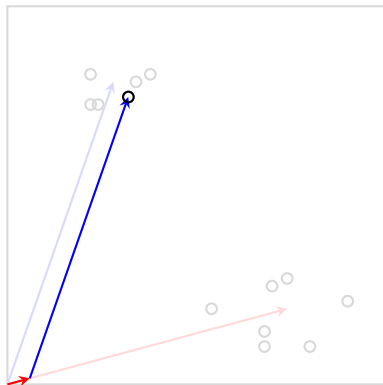
$$\begin{aligned}x &= 1.23 \times \text{red arrow} \\ &+ -0.04 \times \text{blue arrow}\end{aligned}$$

Decomposition



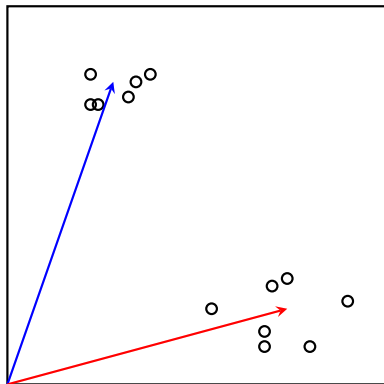
$$\mathbf{x} = 0.14 \times \text{red arrow} + 0.99 \times \text{blue arrow}$$

Decomposition



$$\mathbf{x} = 0.08 \times \text{red arrow} + 0.93 \times \text{blue arrow}$$

Decomposition



$\approx$

U "weights"	V "Components"
0.96 -0.12	
1.23 -0.04	
...	
0.14 0.99	
0.08 0.93	
...	

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set X is considered as a matrix $X \in \mathcal{M}_{N \times d}(\mathbb{R})$,
- We consider decompositions using components $V \in \mathcal{M}_{K \times d}(\mathbb{R})$ and weights $U \in \mathcal{M}_{N \times K}(\mathbb{R})$,
- PCA estimates K components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix XX^T

$$\begin{array}{c} d \\ \hline \begin{array}{|c|} \hline X \\ \hline \end{array} \\ \hline N \end{array} \approx \begin{array}{c} K \\ \hline \begin{array}{|c|} \hline U \\ \hline \end{array} \\ \hline N \end{array} \times \begin{array}{c} \begin{array}{|c|} \hline V \\ \hline \end{array} \\ \hline d \end{array}$$

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set X is considered as a matrix $X \in \mathcal{M}_{N \times d}(\mathbb{R})$,
- We consider decompositions using components $V \in \mathcal{M}_{K \times d}(\mathbb{R})$ and weights $U \in \mathcal{M}_{N \times K}(\mathbb{R})$,
- PCA estimates K components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix XX^T

$$\begin{array}{c} d \\ \hline \begin{array}{|c|} \hline X \\ \hline \end{array} \\ \hline N \end{array} \approx \begin{array}{c} K \\ \hline \begin{array}{|c|} \hline U \\ \hline \end{array} \\ \hline N \end{array} \times \begin{array}{c} \begin{array}{|c|} \hline V \\ \hline \end{array} \\ \hline d \\ K \end{array}$$

Principal Components Analysis

Definitions

Principal components analysis solves the following matrix factorization problem:

- The set X is considered as a matrix $X \in \mathcal{M}_{N \times d}(\mathbb{R})$,
- We consider decompositions using components $V \in \mathcal{M}_{K \times d}(\mathbb{R})$ and weights $U \in \mathcal{M}_{N \times k}(\mathbb{R})$,
- PCA estimates K components that are orthogonal and ordered by importance (variance explained)
- It is based on the Singular Value Decomposition (SVD) of the covariance matrix XX^T

$$\begin{matrix} & d \\ \begin{matrix} \nearrow \\ \searrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \\ N & X \\ \begin{matrix} \nwarrow \\ \swarrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \end{matrix} \approx \begin{matrix} & K \\ \begin{matrix} \nearrow \\ \searrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \\ N & U \\ \begin{matrix} \nwarrow \\ \swarrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \end{matrix} \times \begin{matrix} & d \\ \begin{matrix} \nearrow \\ \searrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \\ K & V \\ \begin{matrix} \nwarrow \\ \swarrow \end{matrix} & \begin{matrix} \text{---} \\ \text{---} \end{matrix} \end{matrix}$$

Principal Components Analysis

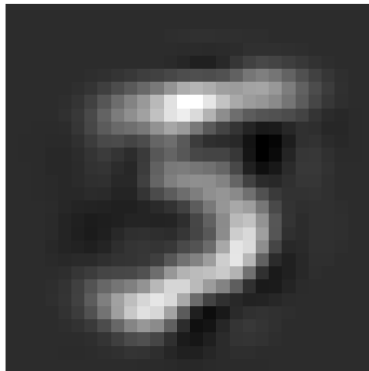
Example of reconstructions on MNIST with $K = 32$



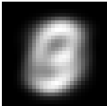
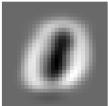
Recall that each image is vectorized, hence each of these images correspond to a row in V .

Principal Components Analysis

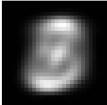
Detailed example of a reconstruction

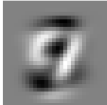


Principal Components Analysis


$$= 122.3 \times$$


Principal Components Analysis


$$= 122.3 \times$$

$$- 316.2 \times$$


Principal Components Analysis

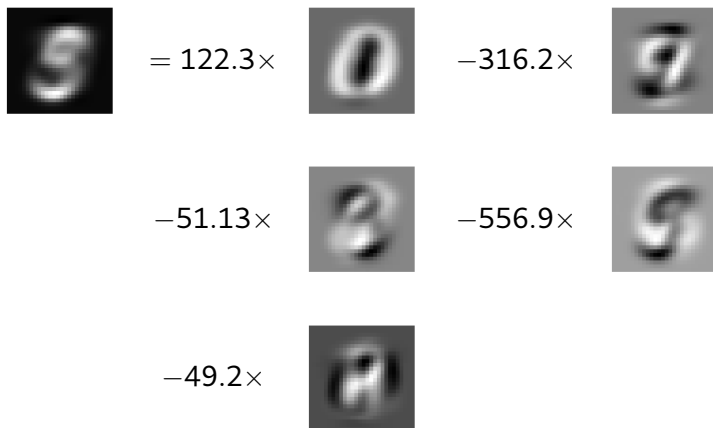
$$\begin{aligned} \text{Image 1} &= 122.3 \times \text{Image 2} - 316.2 \times \text{Image 3} \\ &\quad - 51.13 \times \text{Image 4} \end{aligned}$$

Principal Components Analysis

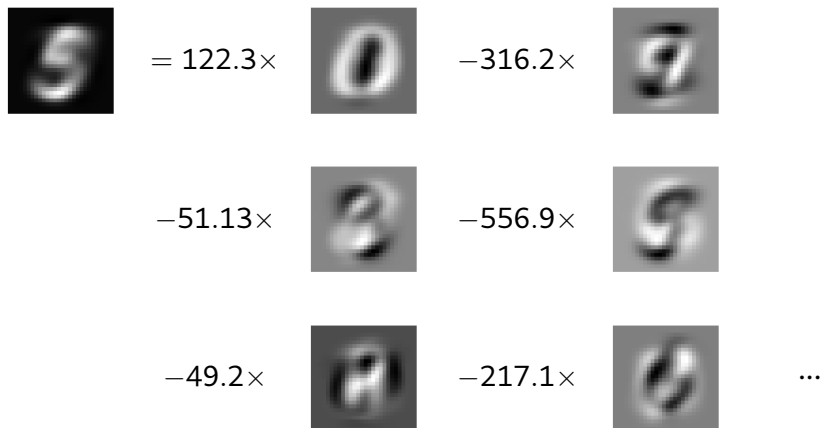
$$\begin{aligned} \text{Image 1} &= 122.3 \times \text{PC1} - 316.2 \times \text{PC2} \\ &- 51.13 \times \text{PC3} - 556.9 \times \text{PC4} \end{aligned}$$

The diagram illustrates the reconstruction of a handwritten digit '5' using the first four principal components (PCs). The original digit is shown on the left. To its right, the equation shows the reconstruction as a linear combination of four principal components, each represented by a grayscale image. The coefficients are: 122.3 for PC1, -316.2 for PC2, -51.13 for PC3, and -556.9 for PC4.

Principal Components Analysis

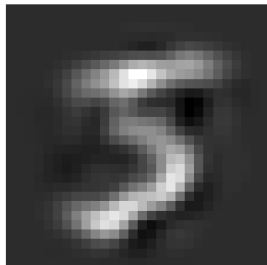


Principal Components Analysis



Principal Components Analysis

Reconstruction with all 32 components:



Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session

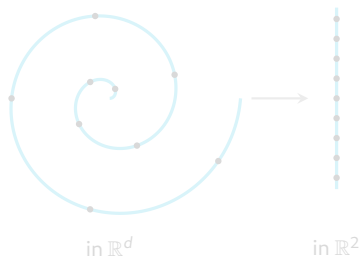
Visualizing high dimensional data

Visualizing high dimensional data helps to **understand its structure**.
But:

- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

Manifold learning

- Hypothesis: high dimensional data lies on (or near) a **lower dimensional** surface.
- Goal: “**unroll**” this surface into 2D/3D, preserving the structure that matters.



The intrinsic dimension can be much smaller than d !

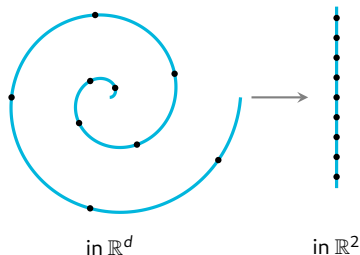
Visualizing high dimensional data

Visualizing high dimensional data helps to **understand its structure**.
But:

- One MNIST digit: $d = 784$
- 4K image: $d \approx 25 \cdot 10^6$.

Manifold learning

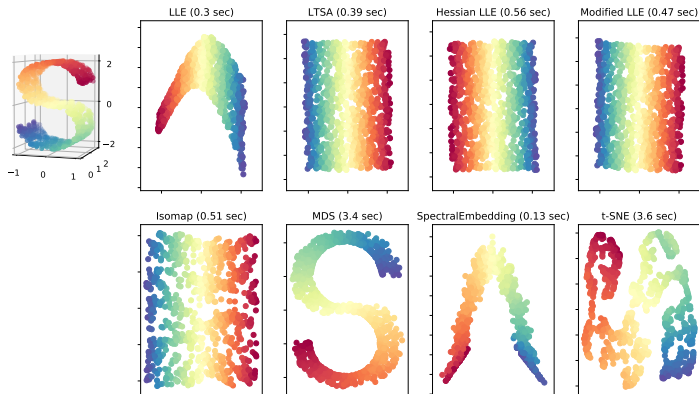
- Hypothesis: high dimensional data lies on (or near) a **lower dimensional surface**.
- Goal: **“unroll”** this surface into 2D/3D, preserving the structure that matters.



The intrinsic dimension can be much smaller than d !

Manifold learning: a family of methods

Manifold Learning with 1000 points, 10 neighbors



Many different approaches exist to uncover lower dimensional structure of high dimensional data. Source : Manifold module, sklearn website

Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

How it works (simplified)

- 1 In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

How it works (simplified)

- 1 In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

Focus: t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding): the most popular method to **visualize** high dimensional data.

How it works (simplified)

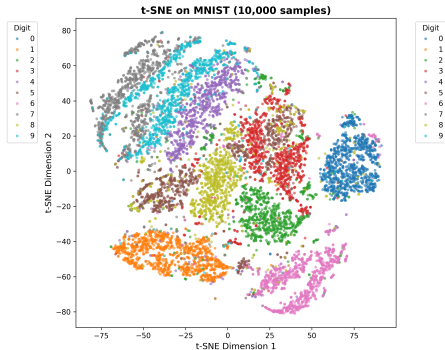
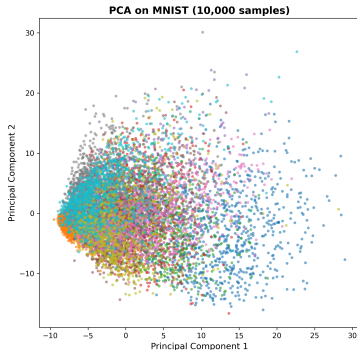
- 1 In $\mathbb{R}^d$: turn distances into probabilities of being neighbors,
- 2 In 2D: define the same kind of distribution for the projected points,
- 3 Move the 2D points to minimize the divergence (Kullback-Leibler) between both.

Caveats

- **Visualization only**: not for data analysis! And cannot project new points,
- Sensitive to hyperparameters,
- **Distances between clusters are not meaningful** (neither are their sizes).

Where PCA preserves global variance, t-SNE preserves local neighborhoods.

PCA vs t-SNE (illustration on MNIST)



Colors = digit classes (**not used** by the algorithms, only for display).

Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models**
- 5 A note on data cleaning
- 6 Lab session

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when they happen to exist*.
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when they happen to exist*.
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when they happen to exist*.
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

How do we evaluate, without ground truth?

Evaluation

- Fully unsupervised (internal metrics). Only use X and the result.
"Is the structure well-formed?"
- Partly supervised: compare to labels, *when they happen to exist*.
"Does the ground-truth structure match what I learned?"
- **Downstream** evaluation: does it help the end task (classification, retrieval, visualization)?

And last but not least: the judgement of a domain expert.

Fully unsupervised clustering metrics

Inertia

The error E of slide 10: sum of squared distances to the centroids.

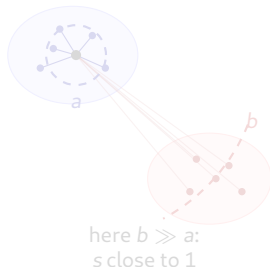
Never increases with K (0 for $K = N$) $\Rightarrow$ never use it alone.

Silhouette score

For each sample: $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$

- a : mean distance to the points of **its own** cluster,
- b : mean distance to the points of the **nearest other** cluster.

$s \approx 1$: well placed; $s \approx 0$: on the border;
 $s < 0$: probably in the wrong cluster.



Fully unsupervised clustering metrics

Inertia

The error E of slide 10: sum of squared distances to the centroids.

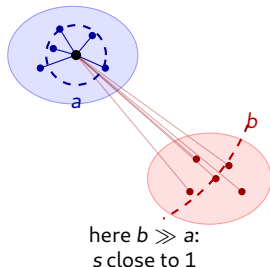
Never increases with K (0 for $K = N$) $\Rightarrow$ never use it alone.

Silhouette score

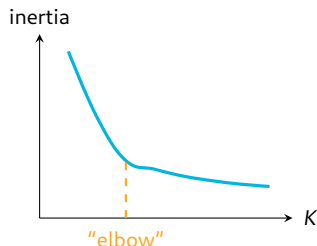
For each sample: $s = \frac{b - a}{\max(a, b)} \in [-1, 1]$

- a : mean distance to the points of **its own** cluster,
- b : mean distance to the points of the **nearest other** cluster.

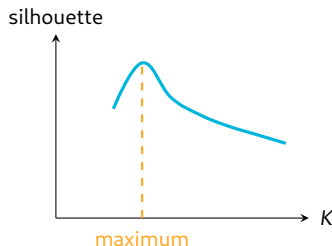
$s \approx 1$: well placed; $s \approx 0$: on the border;
 $s < 0$: probably in the wrong cluster.



Back to choosing K : two heuristics



Elbow: look for the bend



Silhouette: look for the maximum

In practice

- Both are heuristics: they may disagree, and real curves are noisy!
- Additionally, consider the stability across runs/subsamples, and whether the clusters make sense,
- K may be imposed by the application (e.g. 5 customer profiles).

External metrics: when labels do exist

Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels.
Invariant to label permutations;
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



Warning: a low score may mean the clustering found another, valid structure.

External metrics: when labels do exist

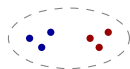
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels.
Invariant to label permutations;
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



homogeneous,
not complete



complete,
not homogeneous

Warning: a low score may mean the clustering found another, valid structure.

External metrics: when labels do exist

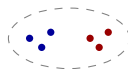
Compare the partition found with a reference partition (the classes). **Never used for training!** Only for validation.

Comparing with ground-truth labels

- **Rand Index:** agreement between the clustering partitions and the labels.
Invariant to label permutations;
- **Homogeneity:** each cluster contains only members of a single class,
- **Completeness:** all members of a class are in the same cluster,
- **V-measure:** harmonic mean of both (the "F1" of clustering).



homogeneous,
not complete



complete,
not homogeneous

Warning: a low score may mean the clustering found another, valid structure.

Evaluating decomposition and manifold learning

Decomposition (PCA, NMF, ...)

- **Reconstruction error** $\|X - UV\|^2$: how much is lost by keeping only K components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- Careful: reconstruction is perfect for $K = d$, but nothing is learned!

Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

Evaluating decomposition and manifold learning

Decomposition (PCA, NMF, ...)

- **Reconstruction error** $\|X - UV\|^2$: how much is lost by keeping only K components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- Careful: reconstruction is perfect for $K = d$, but nothing is learned!

Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

Evaluating decomposition and manifold learning

Decomposition (PCA, NMF, ...)

- **Reconstruction error** $\|X - UV\|^2$: how much is lost by keeping only K components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- Careful: reconstruction is perfect for $K = d$, but nothing is learned!

Manifold learning (t-SNE, ...)

- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

Evaluating decomposition and manifold learning

Decomposition (PCA, NMF, ...)

- **Reconstruction error** $\|X - UV\|^2$: how much is lost by keeping only K components,
- **Explained variance ratio**: share of variance carried by each component; its cumulative sum may be used to choose K ("keep 95%"),
- Careful: reconstruction is perfect for $K = d$, but nothing is learned!

Manifold learning (t-SNE, ...)

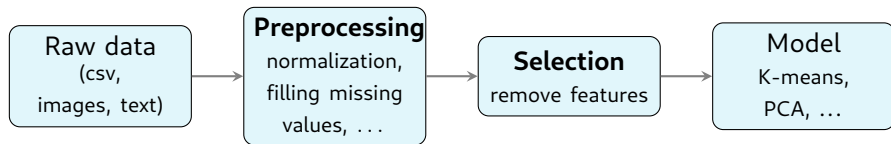
- No reconstruction, hence no natural error,
- In practice: **look at the plot**, color it with any metadata you have, and check that your conclusions survive a change of hyperparameters.

Ultimately: does it help the next step of your project?

Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning**
- 6 Lab session

Features in standard Machine Learning pipelines



Why it matters here

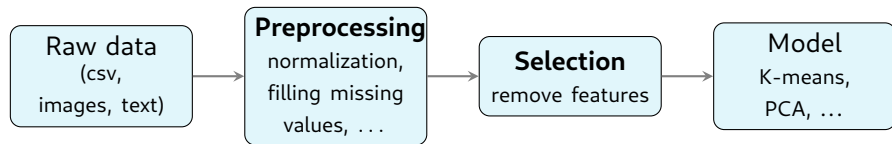
- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

"Garbage in, garbage out"

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised** and **supervised** settings.

Features in standard Machine Learning pipelines



Why it matters here

- Every method seen today relies on **distances** or **variance**,
- Both are **not invariant** to the units and the scale of the features,

"Garbage in, garbage out"

Data preparation may matter more than the model itself!

N.B.: valid in **unsupervised** and **supervised** settings.

Feature preprocessing

Objective: change the values of the features, without removing any.

Classical preprocessing tools

- **Encoding:** turn categories into numbers (OneHotEncoder), discretize continuous values,
- **Scaling / standardization:** same mean and variance for all features (StandardScaler), or same range (MinMaxScaler),
- **Normalization:** put each *sample* on the unit sphere (Normalizer),
- **Power / quantile transforms:** make a skewed feature more Gaussian,
- **Manual feature engineering:** build the feature the model cannot guess (ratios, dates → day of week).

<https://scikit-learn.org/stable/modules/preprocessing.html>

Objective: remove useless features.

Useful for: fighting the curse of dimensionality, speeding up computations, keeping results interpretable, ...

Selecting the features

- Remove features with low variance (`VarianceThreshold`): a constant column carries no information,
- Remove redundant features (strongly correlated with another one).

Note: PCA is *not* feature selection! It builds new features as combinations of all of them.

https://scikit-learn.org/stable/modules/feature_selection.html

Plan

- 1 Clustering
- 2 Decomposition
- 3 Manifold learning
- 4 Evaluating unsupervised models
- 5 A note on data cleaning
- 6 Lab session**

Lab Session 3: Unsupervised Learning

Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with `KMeans`,
- Decompose with `PCA`, visualise with `PCA` and `TSNE`,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore `DBSCAN`, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try `UMAP` for visualisation.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/>

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.

Lab Session 3: Unsupervised Learning

Part 1 — Toy datasets, with `sklearn`

- **Generate toy datasets** (blobs, moons, ...),
- Preprocess/select features, then cluster with KMeans,
- Decompose with PCA, visualise with PCA and TSNE,
- Find **where K-means breaks**, and try a different clustering method fitting the shape of the data.

Then, do the same on a **real dataset**, and explore a different clustering approach.

You may also explore DBSCAN, agglomerative clustering, Gaussian Mixture Models, spectral clustering; you may also try UMAP for visualisation.

Link to the lab: <https://mee-labs.gitlab-pages.imt-atlantique.fr/intro2ai/lab3/>

Part 2 — Your own project data

Apply the very same pipeline to the dataset of **your project**.

⇒ prepare your features and say why, visualise your data in 2D, cluster it and confront the clusters with what you already know. If you have labels, use them to *evaluate*, never to fit.